UAV

Challenge 1 documentation

Flag : KST{1149_62}

Step 0 lab setup :

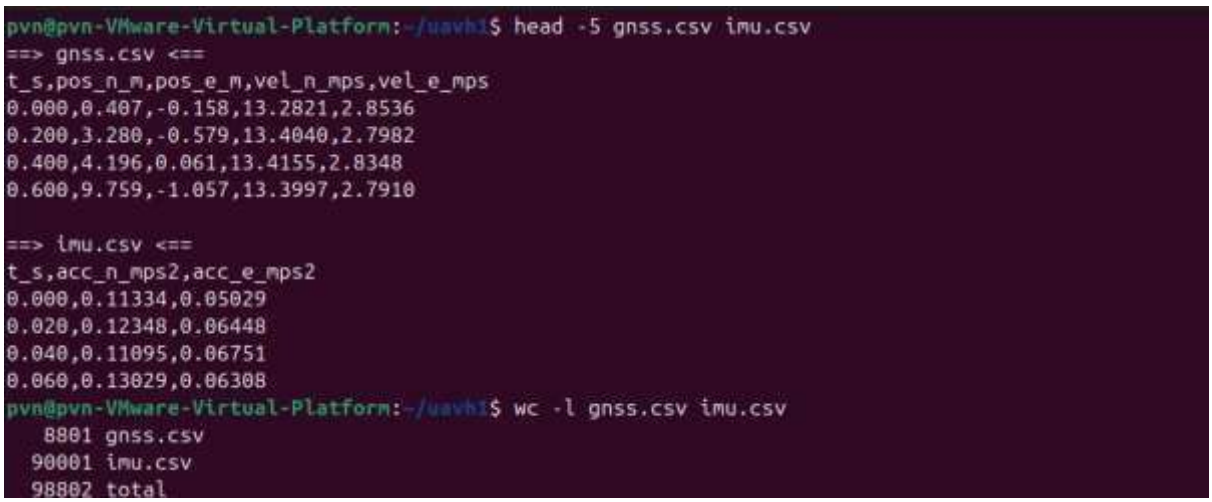






Step 1 :sample inspect :

For sample testing for first 5 rows



#To find gaps in GNSS time (normal step is 0.2 s)



Step 3: the logic Integrates IMU, computes the residual, writes residual.csv and residual.png

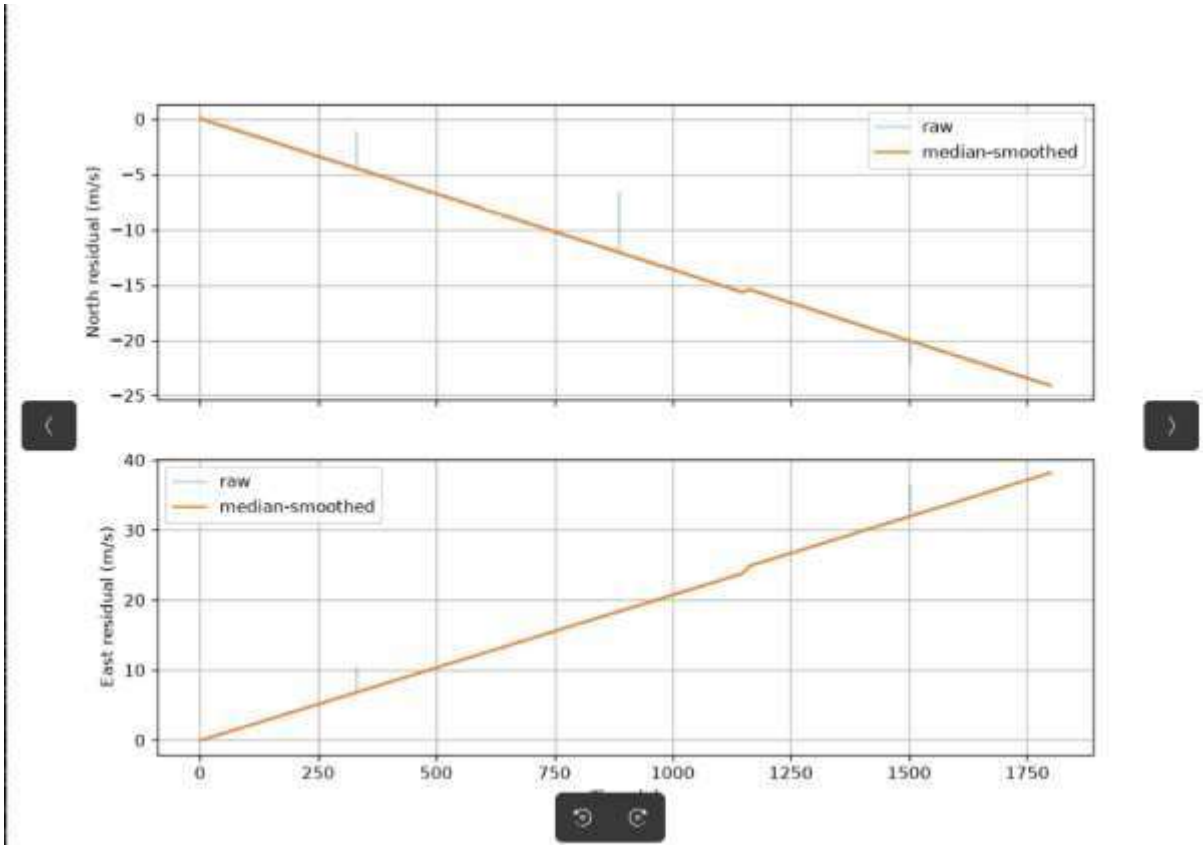
- IMU gives acceleration, so velocity = starting velocity + running sum of (a × 0.02).
- We anchor the start to GNSS velocity at t = 0 (13.2821, 2.8536), since GNSS is clean at the start.
- At each GNSS timestamp we compare: residual = GNSS velocity − IMU velocity.

- Before the attack the residual should be roughly flat or slowly drifting (IMU bias). After the onset it should settle at a new, steady offset.

#for coding I took help of ai for the above logic:

#Residual.py# Output:





Step3 : First Detection Attempt

I wrote it to get a first answer: remove the drift, then use a 5σ threshold to find when the residual leaves the noise. It gave 1150.8 s and 63.4°, but the threshold only fires after the signal is already clear, so it was a rough start.

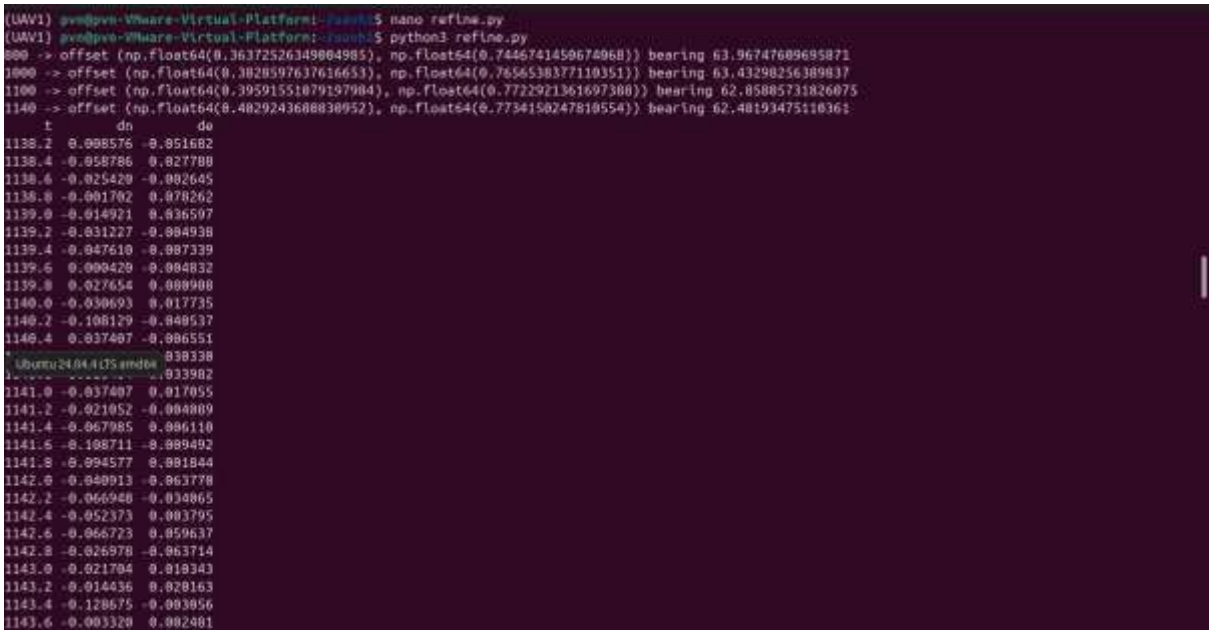
Onset.py:



Step 4: "Checking How Reliable the First Answer Was

I wrote it to test the onset.py result. It showed two problems: the bearing moved from 62.5° to 64° depending on the drift-fit window, and the raw data showed a ramp, not a step. That told us the method needed to change.

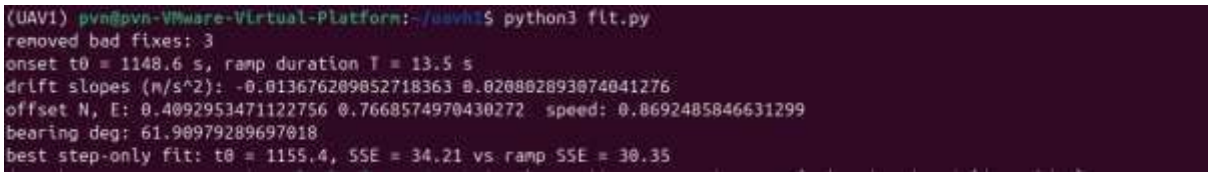
Refine.py



```
1143.8 -0.043604  0.021845
1144.0 -0.050950 -0.046758
1144.2 -0.079070 -0.016224
1144.4 -0.051983 -0.025170
1144.6 -0.061458 -0.036492
1144.8 -0.043295  0.039115
1145.0 -0.023985  0.077726
1145.2 -0.030015  0.050903
1145.4 -0.059653  0.038741
1145.6 -0.022386 -0.006918
1145.8 -0.007721  0.021566
1146.0 -0.049074 -0.015319
1146.2 -0.050104  0.036704
1146.4 -0.074016  0.097651
1146.6  0.021477  0.084376
1146.8 -0.120611  0.088969
1147.0  0.005854 -0.015078
1147.2  0.007451  0.051816
1147.4 -0.005245  0.020390
1147.6 -0.035047  0.025048
1147.8  0.027874  0.004273
1148.0  0.012072  0.037865
1148.2 -0.025592  0.029217
1148.4 -0.074648  0.051453
1148.6 -0.015127  0.085131
1148.8 -0.008587  0.072663
1149.0 -0.017748  0.083575
1149.2  0.033364  0.084135
1149.4 -0.049562  0.055424
1149.6 -0.016595  0.089087
1149.8 -0.010012  0.066308
1150.0  0.046886  0.100883
1150.2  0.092240  0.105358
1150.4  0.036260  0.135390
1150.6  0.001474  0.048192
1150.8  0.067256  0.136375
1151.0  0.079824  0.191534
1151.2  0.078626  0.107984
1151.4  0.010818  0.170292
1151.6  0.123482  0.179993
1151.8  0.035144  0.267765
1152.0  0.066349  0.245421
1152.2  0.068757  0.196975
1152.4  0.094459  0.222898
1152.6  0.042307  0.217231
1152.8  0.091083  0.218417
        0.094536  0.278556
        0.132018  0.291066
1153.4  0.141282  0.325331
1153.6  0.138673  0.315267
1153.8  0.099017  0.312541
1154.0  0.095822  0.239055
1154.2  0.107517  0.321212
1154.4  0.163944  0.321620
1154.6  0.210682  0.347143
1154.8  0.112743  0.304732
1155.0  0.145325  0.322175
1155.2  0.163645  0.370581
1155.4  0.192052  0.390998
1155.6  0.242406  0.424100
1155.8  0.275834  0.409058
1156.0  0.139582  0.469223
1156.2  0.228679  0.429093
1156.4  0.266089  0.540482
1156.6  0.275687  0.482270
1156.8  0.266298  0.481956
1157.0  0.237189  0.497405
1157.2  0.261021  0.476798
```

Terminal





onset t0 = 1148.6 s bearing deg:

61.90979289697018

Step 5:**"Final Joint Fit (Drift + Spoof Ramp)"**
I wrote it to fix both problems from refine.py. It fits the IMU drift and the spoofing ramp together in one model, so there is no extrapolation error. It gives the final onset (1148.6 s) and bearing (61.9°).

The order in one line: detect it (onset), test how far to trust it (refine), then fix the weaknesses and get the final answer (fit).

Fit.py output:

Decisions

1. **Gap at 640 s:** This appears to be a loss of lock rather than a spoofing event. There is no corresponding step in the residual that would indicate spoofing.

2. **Points at 331.4, 887.0, and 1502.2 s:** These are isolated points and are more likely to be bad fixes. They do not show the characteristics of the attack.
3. **IMU drift:** The accelerometer shows a relatively stable bias of approximately **−0.0137 m/s² in the North direction** and **+0.0208 m/s² in the East direction**. This behavior remains consistent across the different methods.
4. **Type of spoofing:** The results are most consistent with a **ramped constant-velocity offset**. The ramp model produced an SSE of **30.35**, which was lower than the step model SSE of **34.21**, making the ramp a better fit.
5. **Attack onset:** I define the start of the attack as **1148.6 s**, which is when the error first begins to increase. This is different from the detection time of **1150.8 s** and the stepequivalent time of **1155.4 s**.
6. **Method used for the final result:** The **joint-fit method** was selected for the final answer because it reduces the extrapolation error observed with the earlier methods.
7. **Bearing:** The joint fit gives a bearing of approximately **61.9°**, while the other methods give values between **62° and 64°**. Therefore, I have rounded the final bearing to **62°**.

Why I used these tools

Shell tools (head, wc, awk): for the first look.
Before analysing anything, I needed to know what the files contained: the columns, the row counts, and whether the timestamps were regular. head shows the structure, wc -l confirms the row counts (8,800 GNSS, 90,000 IMU), and awk scans the whole timestamp column and prints any gap bigger than 0.2 s. These commands run in a second, need no setup, and don't change the original data. They also found the 40.2 s gap at 640 s without a script.

Python (Pandas, NumPy, Matplotlib): for the real analysis.
The core of the task has four parts:

- integrating 90,000 acceleration samples into velocity
- matching the 50 Hz IMU to the 5 Hz GNSS timestamps
- fitting drift and offset together by least squares
- plotting the results

Shell tools can't do these cleanly. Python also makes the work repeatable: anyone can rerun my scripts on the same files and get the same numbers, which a spreadsheet with manual steps can't guarantee.

```
#####onset.py import pandas as
pd, numpy as np

g = pd.read_csv("residual.csv")

# 1. refit drift on a clean section (same as before)
clean = g[g.t_s < 1000] pn = np.polyfit(clean.t_s,
clean.res_n, 1) pe = np.polyfit(clean.t_s,
clean.res_e, 1) g["dn"] = g.res_n - np.polyval(pn,
g.t_s) g["de"] = g.res_e - np.polyval(pe, g.t_s)

# 2. light smoothing only (11 samples = 2.2 s) so the onset isn't blurred
g["dn_s"] = g.dn.rolling(11, center=True, min_periods=1).median() g["de_s"]
= g.de.rolling(11, center=True, min_periods=1).median()

# 3. baseline noise from the clean section base =
g[g.t_s < 1000] mn, me = base.dn_s.mean(),
base.de_s.mean() sn, se = base.dn_s.std(),
base.de_s.std() print("baseline mean:", mn, me, "
noise std:", sn, se)

# 4. deviation size; first time it stays above 5 sigma for 5 s dev =
np.hypot(g.dn_s - mn, g.de_s - me) thr = 5 * np.hypot(sn, se) above =
(dev > thr).astype(int) sustained = above.rolling(25).sum() == 25      #
25 samples = 5 s at 5 Hz idx = sustained.idxmax() onset_idx = idx - 24
print("onset time:", g.t_s.iloc[onset_idx])

# 5. offset = plateau mean minus baseline mean plateau =
g[(g.t_s > 1200) & (g.t_s < 1700)] on =
```

```
plateau.dn_s.median() - mn oe = plateau.de_s.median() -
me print("offset N, E:", on, oe, " speed:", np.hypot(on, oe))
print("bearing deg:", np.degrees(np.arctan2(oe, on)) % 360)
```

###residual.py

```
import pandas as pd, numpy as np import
matplotlib.pyplot as plt
g = pd.read_csv("residual.csv")

# 1. fit the bias drift (a straight line) on a section assumed clean
clean = g[g.t_s < 1000] pn = np.polyfit(clean.t_s, clean.res_n, 1)
pe = np.polyfit(clean.t_s, clean.res_e, 1) print("drift slopes
(m/s^2):", pn[0], pe[0])

# 2. subtract the drift from the whole flight g["dn"] = g.res_n -
np.polyval(pn, g.t_s) g["de"] = g.res_e - np.polyval(pe, g.t_s)
g["dn_s"] = g.dn.rolling(51, center=True, min_periods=1).median()
g["de_s"] = g.de.rolling(51, center=True, min_periods=1).median()

# 3. plot fig, ax = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
ax[0].plot(g.t_s, g.dn, alpha=0.3); ax[0].plot(g.t_s, g.dn_s) ax[0].set_ylabel("N
detrended (m/s)"); ax[0].grid() ax[1].plot(g.t_s, g.de, alpha=0.3);
ax[1].plot(g.t_s, g.de_s) ax[1].set_ylabel("E detrended (m/s)");
ax[1].set_xlabel("Time (s)"); ax[1].grid() plt.savefig("detrended.png", dpi=120)
```

```
# 4. print the region around the suspected onset
print(g[(g.t_s > 1100) & (g.t_s <
1260)][["t_s","dn_s","de_s"]].iloc[::25].to_string(index=False))



#refine.py import pandas as pd,
numpy as np


g = pd.read_csv("residual.csv")


def run(fit_end):
   clean = g[g.t_s < fit_end]     pn = np.polyfit(clean.t_s,
clean.res_n, 1)     pe = np.polyfit(clean.t_s, clean.res_e, 1)
dn = g.res_n - np.polyval(pn, g.t_s)     de = g.res_e -
np.polyval(pe, g.t_s)     pl = (g.t_s > 1200) & (g.t_s < 1700)
on, oe = dn[pl].median(), de[pl].median()     return dn, de, on,
oe, np.degrees(np.arctan2(oe, on)) % 360


# 1. sensitivity of bearing to the drift-fit window for
fe in (800, 1000, 1100, 1140):
   print(fe, "-> offset", run(fe)[2:4], "bearing", run(fe)[4])


# 2. raw look around the onset (no smoothing) dn, de, on, oe, b = run(1100) w =
(g.t_s > 1138) & (g.t_s < 1172) print(pd.DataFrame({"t": g.t_s[w], "dn": dn[w], "de":
de[w]}).to_string(index=False))


# 3. plateau flatness pl = (g.t_s > 1200) & (g.t_s
<            1700)            print(dn[pl].describe());
print(de[pl].describe())     #     4.     bad     fixes
```

```
print(g[(dn.abs()  >  1)  |  (de.abs()  >
1)][["t_s"]].head(20))



####### fit.py import pandas as
pd, numpy as np

g = pd.read_csv("residual.csv")

# drop bad fixes: points far from a local median for
c in ("res_n", "res_e"):
   g[c + "_m"] = g[c].rolling(51, center=True, min_periods=1).median() ok =
((g.res_n - g.res_n_m).abs() < 0.5) & ((g.res_e - g.res_e_m).abs() < 0.5) d =
g[ok] t = d.t_s.values print("removed bad fixes:", (~ok).sum())

def ramp(t, t0, T):
if T == 0:
      return (t >= t0).astype(float)
return np.clip((t - t0) / T, 0, 1)

def fit(t0, T):
   s = ramp(t, t0, T)
   A = np.column_stack([np.ones_like(t), t, s])
out, sse = [], 0  for y in (d.res_n.values,
d.res_e.values):      coef, *_ =
np.linalg.lstsq(A, y, rcond=None)      sse +=
((y - A @ coef) ** 2).sum()
out.append(coef)  return sse, out
```

```
best = None for t0 in
np.arange(1140, 1160, 0.2):     for T
in np.arange(0, 25, 0.5):
       sse, co = fit(t0, T)         if best is
None or sse < best[0]:
           best = (sse, t0, T, co)


sse, t0, T, co = best cn, ce = co[0][2], co[1][2] print("onset
t0 = %.1f s, ramp duration T = %.1f s" % (t0, T)) print("drift
slopes (m/s^2):", co[0][1], co[1][1]) print("offset N, E:", cn,
ce, " speed:", np.hypot(cn, ce)) print("bearing deg:",
np.degrees(np.arctan2(ce, cn)) % 360)

# for comparison: best pure step (T = 0) best0 = min(((fit(t0, 0)[0], t0) for t0 in
np.arange(1140, 1160, 0.2))) print("best step-only fit: t0 = %.1f, SSE = %.2f vs ramp SSE =
%.2f" % (best0[1], best0[0], sse))
```